

# Achieving MC/DC using UML Communication Diagram

Parbati Mahanto  
Dept. of Computer Science and Engg.  
N.I.T. Rourkela  
Odisha, India  
parbati1906@gmail.com

Swadhin Kumar Barisal  
Dept. Computer Science and Engg.  
N.I.T. Rourkela  
Odisha, India  
517CS8006@nitrkl.ac.in

Durga Prasad Mohapatra  
Dept. Computer Science and Engg.  
N.I.T. Rourkela  
Odisha, India  
durga@nitrkl.ac.in

**Abstract**—Today, in order to reduce the burdensome task of software testing, test case generation has been elongated and can be generated not only from the coding documents but also from Software Requirement Specification (SRS) and Object-Oriented design models (e.g. UML model). A Unified Modeling Language (UML) contains different diagrams for visual modeling of the Object-Oriented software. To describe individual perspectives of a software system different types of UML diagrams are used. In this study, we have used UML Communication Diagram and generated test cases by considering Modified Condition/Decision Coverage (MC/DC) as a precedent. A UML Communication diagram is used to describe a set of objects collaborating to implement a specific task and illustrate the relationship between them through a complex operation. We propose an automated model-based technique for MC/DC test case generation from the UML Communication Diagram and for that, we have proposed a framework named MC/DC Estimator for UML Communication Diagram (MEUCD) which is a combination of different tools like Modelio, JiBX, jCUTE, and COPECA. We first design a UML Communication Diagram in Modelio UML Modeler and then, by using the classes and instances of the communication diagram, we design the XML Schema Design (XSD) model in Modelio XSD Designer. Then, the XSD is given as input to a Java XML Binding tool JiBX to generate the Java Code. Next, we convert this Java code to a jCUTE executable form and supply it to jCUTE for generating test cases. Finally, MC/DC coverage is measured by using the acceptable Java code and the generated test cases of jCUTE.

**Index Terms**—MC/DC; UML Communication diagram; Concolic Testing.

## I. INTRODUCTION

Software testing is crucial and most difficult part of software development. The total cost of testing is 50% [1] of the overall project costs. Software testing can be done in two ways: Manual Testing and Automated Testing. But for Object-Oriented programs, manual testing is time taking and error-prone. Hence Automated testing has been carried out to reduce the testing time, effort and error as compared to manual testing. UML diagrams are used to understand the software system better through a visual representation of the system. There are different UML diagrams like Use case diagram, Class diagram, Activity diagram, Sequence diagram, Communication diagram

etc. in order to explain behavioral, structural and interaction views of the system. Generally during software development test cases are designed from source code [4]. But we can also design test cases from UML diagrams at design phase in order to achieve control over testing phase at early stage of software development. Hence testing effort and time will reduce. In this work, we have used a UML Communication diagram for generating test cases and measuring MC/DC percentage of the test cases. UML Communication diagram has both structural and behavioral aspects of Object-Oriented System [10]. Structural aspect represents the objects and the association between the objects while the behavioral aspect shows the flow of control between the objects as sequence of messages. Because of the behavioral aspect of Object-Oriented System an UML Communication Diagram is able to show the conditions.

In white box testing, code coverage is an important criteria as it checks the quality of the test suite and MC/DC is one of the strongest techniques among all code coverage based techniques. MC/DC is a mandatory criteria for Level A certification in safety critical software applications by the guidelines provided by RTCA/DO2-178B [2] and DO-178C [3] standards. System failure in safety critical system can cause life disaster. Therefore, for effective test data generation of MC/DC we need a rigorous and thorough testing process. We can use Concolic Testing for MC/DC test case generation as it combines both random testing and symbolic testing.

The objective of the presented paper is to propose an automatic approach to generate MC/DC test cases from UML Communication Diagram and measure its Modified Condition / Decision Coverage(MC/DC). The presented approach MC/DC Estimator for UML Communication Diagram(MEUCD) focuses on automatic model-based Modified Condition / Decision Coverage(MC/DC) testing with UML communication diagram. MEUCD integrates four different tools: MODELIO<sup>1</sup>, JiBX<sup>2</sup>, jCUTE<sup>3</sup>, and COPECA.

The remaining part of the article is organized as follows: Section II describes the Basic Concepts, Section III describes the Proposed Approach, Section IV shows Experimental results, Section V discusses Comparison with related work and Section VI concludes the paper with suggestions for Future work.

<sup>1</sup><https://www.modelio.org/>

<sup>2</sup><https://www.jibx.sourceforge.net/>

<sup>3</sup><https://www.osl.cs.illinois.edu/software/jcute/>

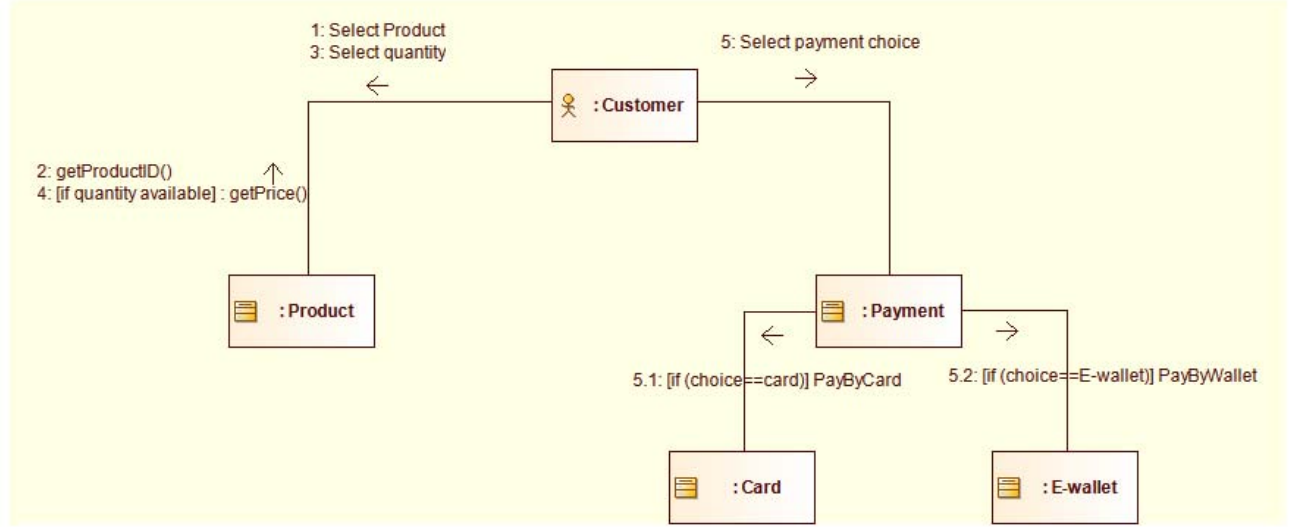


Fig. 1. A Communication diagram for Online card payment

## II. BASIC CONCEPTS

### A. UML Communication Diagram

A UML Communication Diagram (earlier known as a Collaboration Diagram in UML 1.x), has both structural and behavioral aspects of an Object-Oriented system through object relationships. This interaction diagram shows a combined information of Class Diagram and Sequence Diagram. In UML Communication Diagrams, objects are connected with associations. Every association is added with a message and shown as a short arrow which points to the direction of message flow. The flow of messages is shown through a numbering sequence. For example, let us consider the Communication diagram for Online card payment shown in Figure 1. Here a customer pays the price of a product by using a card or by E-wallet. In this scenario, we have 1 actor and 4 objects involved. Total 4 associations present between objects with total seven messages and 3 guard conditions.

### B. Modified Condition/Decision Coverage (MC/DC)

Modified Condition/Decision Coverage (MC/DC) mainly concentrates on the distinctive impact of individual condition on the result of whole condition. For this, the result of the whole condition must change if the value of one individual condition is changed and all other values of condition remain stable. For example, let us consider A&&B. Then following steps will be taken for generating MC/DC test cases:

- Generate the Truth Table for the given predicate.
- Produce the Extended Truth Table(ETT) that shows the test pairs which can independently affect the decision of predicate. The Extended Truth Table(ETT) is shown in Table 1. Here T denotes True and F denotes False.
- Show the independence of A:(1,3) and B:(1,2).

- Get the resulting MC/DC test cases:(1,2,3). Hence MC/DC test suite is TS= { TT, TF, FF }.

TABLE I  
EXTENDED TRUTH TABLE FOR A&&B

| S.N | A | B | A && B | A | B |
|-----|---|---|--------|---|---|
| 1   | T | T | T      | 3 | 2 |
| 2   | T | F | F      |   | 1 |
| 3   | F | T | F      | 1 |   |
| 4   | F | F | F      |   |   |

### C. Concolic Testing

Concolic testing is a hybrid testing technique that combines both random testing(or, concrete execution) and symbolic testing (or, symbolic execution) for automated test case generation [11]. For test case generation, Concolic testing explores all the possible execution paths of a program. Then, it ensures that all reachable codes are executed including any assertion violation. For example, let us consider the following program segment:

- 1) var x=readinput();
- 2) var r=readinput();
- 3) x=2\*x;
- 4) if (x > 100)
- 5) if (r==1)
- 6) r=1.3;
- 7) halt

Figure 2 shows Execution Tree of the above program during concolic testing. During concolic testing, we can see that all the execution tree explores all possible paths of the program. Here for our example, we have three possible execution paths.

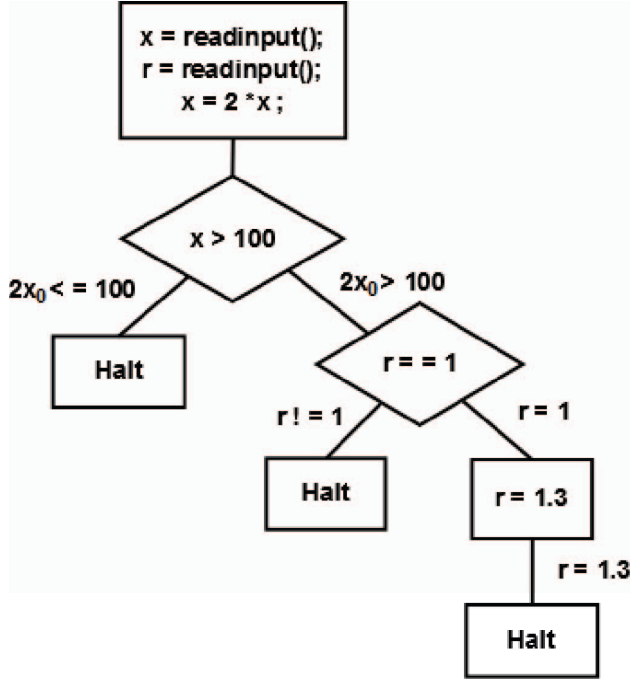


Fig. 2. Execution Tree of the example program using concolic testing

### III. PROPOSED APPROACH

We have suggested an approach for generating MC/DC test cases and measuring MC/DC coverage of these test cases and named it MC/DC Estimator for UML Communication Diagram (MEUCD). In this section, first we discuss the overview of proposed framework. Then, we discuss proposed approach in detail and then Algorithmic description of MEUCD at the end.

#### A. Overview

Figure 3 shows the framework for the MC/DC Estimator for UML Communication Diagram (MEUCD). MEUCD has four modules and they are MODELIO, JiBX, jCUTE, and COPECA. MODELIO, JiBX, and jCUTE are open source tools while COPECA is an in-house developed tool. By using MODELIO tool, a Communication Diagram for a particular use case is designed first. Then, XML Schema Design (XSD) document of the Communication diagram is generated from MODELIO tool. Now this XSD code is given as input to JiBX, which converts the XSD code to Java code. Java code is modified into jCUTE executable Java program form and given to jCUTE tool. After supplying this Java program to jCUTE, it produces test cases automatically. For calculating MC/DC, these test cases are supplied along with the Java program to COPECA. COPECA computes the MC/DC and gives the result in form of MC/DC %.

#### B. Detailed Description

Here, framework of MEUCD is discussed in detail and also, each of the modules of MEUCD is discussed.

At first, a UML Communication diagram is drawn using Modelio's UML Modeler. Modelio is an open source tool and can also be used to draw different UML diagrams like Use case, Class, Sequence, Communication, Activity, State machine, Object and Deployment diagrams. In Figure 3, Modelio uses its UML Modeler to design Communication diagram. After designing the Communication diagram, objects involved in the scenario are used to design the XSD Model. The XSD Model is modeled in Modelio XSD designer using the classes involved in the Communication diagram. In Modelio XSD Designer, a XSD model contains the objects of communication diagram as Complex Type element for the XML Schema and it also shows the associations between these Complex Type elements. By using the designed XSD model, a XSD file is generated from Modelio XSD Designer. This XSD code needs some manual modifications if there are guard conditions in communication diagram. For generating Java code from XSD code, we need a Java XML Schema Binder. Here, we have used JiBX open source tool for generating Java code from our XSD code. JiBX binds XML data to Java objects. For building objects from an XML Schema, JiBX calls unmarshalling method and for generating XML Schema from Java objects it calls marshalling method. Hence, Java code is generated from XSD code using JiBX. This Java code contains simple conditions, so it is manually modified to predicates for generation of MC/DC test cases. Then, the Java code is given to jCUTE which is used to generate test cases. jCUTE uses Concolic Testing to generate test cases where it uses execution tree and explores all possible execution paths for all possible input values. Now, for evaluating the MC/DC coverage these generated test cases and the Java Program are supplied to MC/DC coverage measuring tool COPECA. COPECA uses the Extended Truth Table (ETT) concept for measuring MC/DC coverage. For measuring MC/DC percentage, COPECA uses a formula which needs the number of independently affected conditions and the total number of conditions present in the Java program. Finally, MC/DC percentage is computed for UML Communication diagram using MEUCD.

#### C. Algorithmic Description

The procedure of MC/DC Estimator for UML Communication Diagram (MEUCD) is shown in Algorithm 1. Line 1 represents that first the UML Communication diagram is designed with the use of UML Modeler of Modelio. Line 2 generates XSD model of Communication diagram using Modelio's XSD designer. Then, XSD code is generated from XSD model by using Modelio's XSD designer in Line 3. Line 4 shows that JiBX is used to produce generic Java code from XSD. Line 5 shows MC/DC test case generation from executable Java code using jCUTE. Line 6 shows the formula for MC/DC computation used by COPECA and for that COPECA needs the Java program and the test cases.

### IV. EXPERIMENTAL RESULTS

In this section required setup and experimental results of our experiment are discussed.

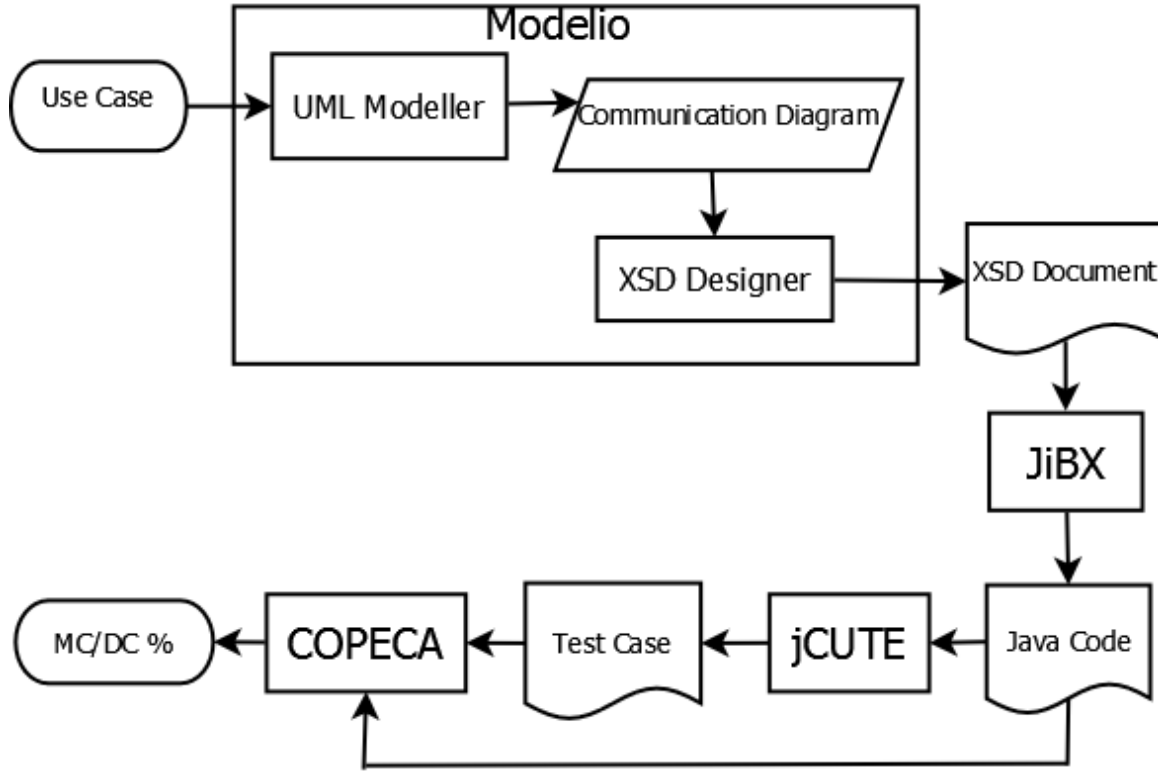


Fig. 3. Framework of MEUCD

#### Algorithm 1 MEUCD

Data: UML Communication diagram.

Result: MC/DC coverage in percentage.

- 1: Prepare a Communication diagram using UML Modeler of Modelio.
- 2: Design the XSD model of the UML Communication diagram using XSD Designer of Modelio.
- 3: Produce XSD code of the XSD model from XSD Designer of Modelio.
- 4: Execute XSD code in JiBX and generate an executable Java code.
- 5: Generate MC/DC test cases in jCUTE for the executable Java code.
- 6: Compute the MC/DC coverage in terms of percentage using COPECA by considering the generated MC/DC test cases and the Java code using the following formula:

$$MC/DC \text{ Coverage} = (IC/TC) * 100 \quad (1)$$

Here, IC is the number of independently affected conditions and TC is the total number of conditions present in Java code.

#### A. Setup

In this section, we present the configuration of our system and details of the tools used.

##### 1) System Configuration:

- Intel(R) i5 version dual core processor with 3.20 GHz processing speed.
- 4 GB RAM.
- Windows 7 Professional of 32 bit as Operating System.

##### 2) Details of Tools used:

- Modelio : version 3.5.0, open source, compatible with java version 1.6 or higher and size 230MB.
- JiBX: open source, works on ANT build IDE, compatible with Java version 1.3 or higher.
- jCUTE: open source, compatible with Java version 1.6 or higher and size 230MB.
- COPECA : in-house developed tool, works on NetBeans IDE and compatible with any Java version.

#### B. Result

We performed our experiment with three case studies which are shown in Table II. For each case study, we have different use case scenarios which are shown in third column. Fourth column to eighth column show the number of Actors involved, numbers of Objects generated, number of Associations

TABLE II  
FEATURES OF UML COMMUNICATION DIAGRAMS USED

| Sl. No. | Case Study | Use Case Scenario   | No. of actors | No. of objects | No. of associations | No. of messages | No. of guard conditions |
|---------|------------|---------------------|---------------|----------------|---------------------|-----------------|-------------------------|
| 1       | OOS        | Online Card Payment | 1             | 4              | 4                   | 7               | 3                       |
| 2       | OOS        | Log in              | 1             | 3              | 5                   | 6               | 1                       |
| 3       | OOS        | Place Order         | 1             | 3              | 3                   | 6               | 2                       |
| 4       | OOS        | Order Delivery      | 1             | 3              | 5                   | 6               | 4                       |
| 5       | OOS        | View Items          | 1             | 5              | 5                   | 8               | 4                       |
| 6       | LMS        | Issue Book          | 1             | 3              | 3                   | 8               | 2                       |
| 7       | LMS        | Return Book         | 1             | 3              | 4                   | 6               | 2                       |
| 8       | LMS        | Renew Book          | 1             | 4              | 4                   | 11              | 2                       |
| 9       | LMS        | Member Login        | 1             | 2              | 2                   | 4               | 1                       |
| 10      | LMS        | Book Reservation    | 2             | 3              | 4                   | 5               | 3                       |
| 11      | ATM        | User Validation     | 1             | 2              | 2                   | 4               | 1                       |
| 12      | ATM        | Withdraw money      | 1             | 3              | 3                   | 10              | 2                       |
| 13      | ATM        | Transfer Money      | 2             | 3              | 3                   | 8               | 3                       |
| 14      | ATM        | Deposit Money       | 2             | 3              | 4                   | 6               | 3                       |
| 15      | ATM        | Balance Inquiry     | 1             | 2              | 4                   | 4               | 1                       |

TABLE III  
RESULTS OBTAINED FROM COPECA

| Sl. No. | Case Study | Use Case Scenario | No. of test-cases generated | No. of independently affected conditions (IC) | Total No. of Conditions (TC) | MC/DC coverage |
|---------|------------|-------------------|-----------------------------|-----------------------------------------------|------------------------------|----------------|
| 1       | OOS        | Card Payment      | 7                           | 3                                             | 6                            | 50%            |
| 2       | OOS        | Log in            | 2                           | 3                                             | 4                            | 75 %           |
| 3       | OOS        | Place Order       | 6                           | 2                                             | 6                            | 33.33 %        |
| 4       | OOS        | Order Delivery    | 6                           | 1                                             | 4                            | 25 %           |
| 5       | OOS        | View Items        | 2                           | 0                                             | 2                            | 0 %            |
| 6       | LMS        | Issue Book        | 6                           | 4                                             | 6                            | 66.66 %        |
| 7       | LMS        | Return Book       | 6                           | 4                                             | 6                            | 66.66 %        |
| 8       | LMS        | Renew Book        | 7                           | 2                                             | 4                            | 50 %           |
| 9       | LMS        | Member Login      | 4                           | 2                                             | 2                            | 100%           |
| 10      | LMS        | Reserve Book      | 5                           | 2                                             | 3                            | 66.66%         |
| 11      | ATM        | User Validation   | 4                           | 1                                             | 2                            | 50 %           |
| 12      | ATM        | Withdraw money    | 7                           | 4                                             | 6                            | 66.66%         |
| 13      | ATM        | Transfer Money    | 6                           | 3                                             | 6                            | 50 %           |
| 14      | ATM        | Deposit Money     | 5                           | 2                                             | 4                            | 50 %           |
| 15      | ATM        | Balance Inquiry   | 6                           | 3                                             | 6                            | 50 %           |

between objects, number of Messages passed and number of Guard conditions present in their UML Communication diagrams respectively.

Table III shows the results obtained from COPECA for the case studies where fourth column presents the number of test cases produced from jCUTE. Fifth column presents the total number of independently affected conditions in the Java code. Sixth column presents the total number of simple conditions in the Java code. Seventh column uses the value of fifth and sixth column for calculating MC/DC coverage in percentage.

After getting all the results, analysis of results is done and shown in Figure 4 and Figure 5. Figure 4 shows the comparison for number of test cases generated for each UML Communication Diagram. Figure 5 shows the comparison between the number of independently affected conditions and total number of simple conditions present in the Java code.

## V. COMPARISON WITH RELATED WORK

Godbole et al. [7] have proposed an approach for automatic generation of MC/DC test cases using concolic testing at



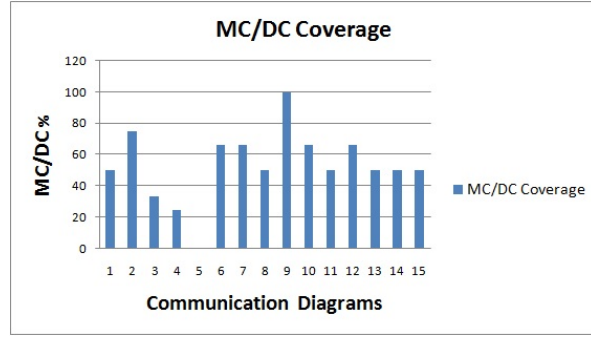


Fig. 4. Test case generated for each Communication Diagram

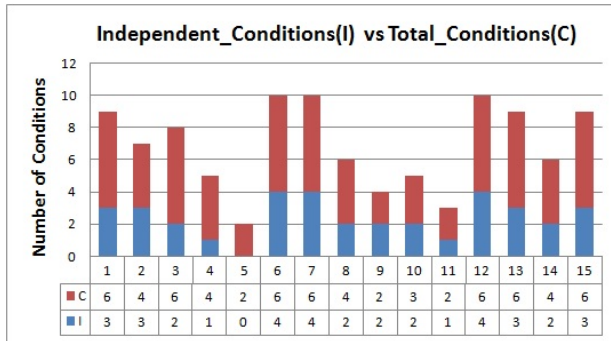


Fig. 5. Comparison between Number of independently affected conditions and Number of simple conditions

design phase. They have used UML sequence diagram for their approach. But in our approach, we have used UML Communication diagram for generation of MC/DC test cases automatically. They have used ArgoUML tool for designing sequence diagram and generated XMI code from the sequence diagram. In our approach, we have used Modelio tool for designing UML Communication diagram and generate XSD model from the Communication diagram. They have used JAXB for Java code generation from XMI code while we have used JiBX for Java code generation from XSD code.

Das et al. [8] have proposed an approach for MC/DC test case generation automatically by using code transformation technique for achieving higher MC/DC coverage. In our approach, we used a new transformation technique where we combined different tools for transformation of communication diagram to Java code and to generate MC/DC test cases from this Java code automatically. They have used CREST concolic tester as automatic generation tool of MC/DC test cases, while we have used jCUTE concolic tester for the same. By applying their code transformation technique, it is also found that MC/DC coverage has increased while in our approach we simply focus on MC/DC test case generation and measured MC/DC coverage only.

Haque et al. [9] have developed an automated tool named MC/DC GEN. It is a web based tool with pair-wise testing technique and takes MC/DC predicates as input and generates

test case pairs for that predicate. After generating test case pairs of each predicate for final test case pairs redundant test case pairs are removed. In our approach, we have developed a framework which combines different automated tools for MC/DC test case generation.

## VI. CONCLUSION AND FUTURE WORK

We have developed an approach for automatic MC/DC test cases generation using UML Communication diagram. We have also measured the MC/DC% of these test cases on the Java program generated from MEUCD. We have experimented with three case studies and these case studies contain fifteen communication diagrams. We have achieved an average of 53.33% MC/DC for fifteen communication diagrams.

We will extend our proposed work in future for calculating some other coverage metrics using some other UML diagrams.

## REFERENCES

- [1] Y. G. Kim, H. S. Hong, D. H. Bae and S. D. Cha, "Test cases generation from UML state diagrams," in *IEE Proceedings-Software*, Aug. 1999, vol.146, no.4, pp.187-192.
- [2] RTCA (Firm), SC 167, *Software considerations in airborne systems and equipment certification*, Washington, D.C. : RTCA, Inc., 1992.
- [3] L. Rierson , *Developing safety-critical software: a practical guide for aviation software and DO-178C compliance*, CRC Press, 2013.
- [4] A. Abdurazik, and J. Offutt, "Using UML collaboration diagrams for static checking and test generation," in *Proceedings of International Conference on the Unified Modeling Language*, October 2000, Springer, Berlin, Heidelberg, pp.383-395.
- [5] J. Rumbaugh, G. Booch, and I. Jacobson, *The unified modeling language reference manual*, Addison Wesley, 1999.
- [6] W. E. Wong, J. R. Horgan, S. London, and H. Agrawal, "A study of effective regression testing in practice," in *Proceedings of The Eighth International Symposium on Software Reliability Engineering*, November 1997, pp.264-274.
- [7] S. Godbole, A. Dutta, A. Das, and D.P. Mohapatra, "Measuring MC/DC at design phase using UML sequence diagram and concolic testing," in *Proceedings of IEEE Annual India Conference (INDICON)*, December 2016, pp. 1-6.
- [8] A. Das, and R. Mall, "Automatic generation of MC/DC test data," *International Journal of Software Engineering Acta Press*, vol.2, no.1, 2013.
- [9] A. Haque, I. Khalil, and K. Z. Zamli, "An Automated Tool for MC / DC Test Data Generation," *IEEE Symposium on Computers Informatics (ISCI)*, Kota Kinabalu, Sabah, Malaysia, 2014, pp.130-135.
- [10] P. Samuel, R. Mall, and P. Kanth, "Automatic test case generation from UML communication diagrams," *Information and software technology*, vol.49, no.2, pp.158-171, 2007.
- [11] M. Sarma, D. Kundu, and R. Mall, "Automatic test case generation from UML sequence diagram," in *Proceedings of IEEE International Conference on Advanced Computing and Communications*, December 2007, pp. 60-67.
- [12] K. Sen and G. Agha, "CUTE and jCUTE: Concolic Unit Testing and Explicit Path Model-Checking Tools," in *Proceedings of The 18th International Conference on Computer Aided Verification*, Springer, Berlin, Heidelberg, August 2006, pp.419-423.
- [13] S. Godbole, A. Dutta, and D. P. Mohapatra, "Reduced energy consumption for MC/DC testing," *International Journal of Business Information Systems*, vol.28, no.4, 2018, pp.447-467.
- [14] S. Godbole, A. Dutta, D.P. Mohapatra, and R. Mall, "Scaling modified condition/decision coverage using distributed concolic testing for Java programs," *Computer Standards Interfaces*, vol.59, August 2018, pp.61-86.